

Kubernetes Deployment and Autoscaling on Amazon EKS

Scenario:

ShopEase is a growing e-commerce company that's getting ready to modernize the way its backend services run. The engineering team wants to nail down a consistent, repeatable way to package and deploy applications before going all-in on microservices at scale.

Right now, the backend service is:

- Running locally on individual machines during development
- Being packaged and started manually every single time
- Behaving differently across machines — what works on one doesn't always work on another
- Running with no orchestration platform watching over it

As the team gears up for future growth, they need a deployment model that's reliable, repeatable, and works the same way regardless of where it runs. One that can also scale when traffic picks up.

That's why the decision was made to move the service to Kubernetes — specifically Amazon Elastic Kubernetes Service (EKS) — to handle container management, deployments, and networking.

My Role as the DevOps Engineer:

My job here is to take this backend application and deploy it the cloud-native way using Kubernetes.

I'm responsible for:

- Packaging the application as a Docker container
- Storing the image in a container registry

- Running the application on a managed Kubernetes cluster
- Exposing the service so it can be accessed from outside
- Understanding how Kubernetes actually manages Pods and routes traffic

Solution:

I'll deploy a containerized backend service on Amazon EKS and let Kubernetes take care of managing it.

The solution includes:

- A Dockerized application stored in Amazon ECR
- An Amazon EKS cluster with EC2 worker nodes
- A Kubernetes Deployment to run and manage Pods
- A Kubernetes Service (NodePort) to expose the application externally
- Health endpoints to verify the application is running correctly

The focus of this project is on getting comfortable with Kubernetes fundamentals — not production-grade optimizations. Things like HTTPS, Ingress, and autoscaling are intentionally left for later projects.

About the Project:

In this hands-on project, I'll be deploying a real backend service on Kubernetes and actually understanding how it runs inside an EKS cluster.

By the end, I'll have learned:

- How application code gets turned into a Docker image
- How Kubernetes pulls images from a registry
- How Deployments create and manage Pods
- How Services route traffic to the right Pods

- How AWS networking and security groups tie into all of this

The application has API endpoints, health checks, and a simple UI to observe what's happening at runtime.

The goal is for Kubernetes to feel practical and logical by the end — not abstract or confusing.

Steps Followed:

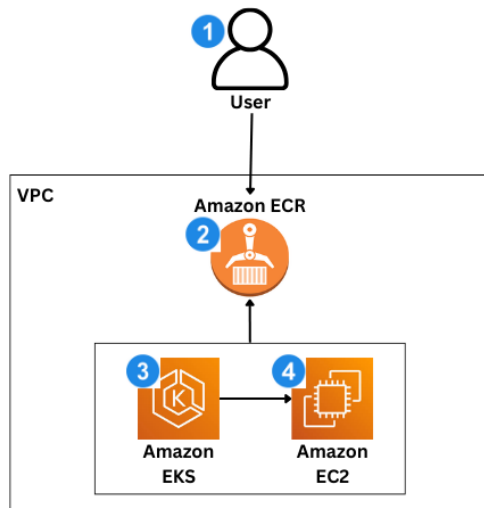
1. Understand Kubernetes and Amazon EKS concepts
2. Containerize the application using Docker
3. Push the Docker image to Amazon ECR
4. Create an Amazon EKS cluster
5. Deploy the application using a Kubernetes Deployment
6. Expose the application using a Kubernetes Service (NodePort)

Services used:

- **Docker** — to containerize the application
- **Amazon ECR** — to store the Docker images
- **Amazon EKS** — the managed Kubernetes cluster
- **Amazon EC2** — the worker nodes that run inside the cluster
- **Amazon VPC** — handles networking for EKS
- **AWS IAM** — manages permissions and access control
- **Kubernetes (kubectl)** — to deploy and manage workloads

Architectural Diagram:

Container Deployment Architecture Using Amazon EKS



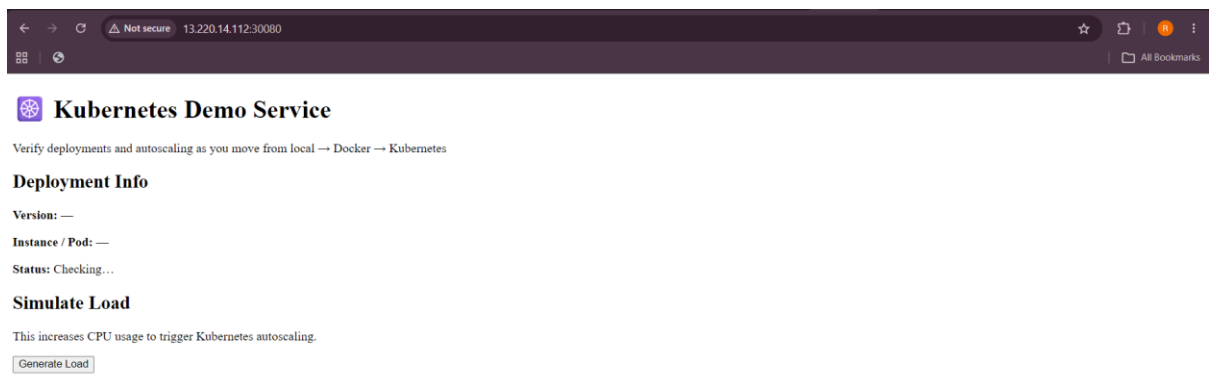
1. **User** accesses the application endpoint.

2. Container images are stored in **Amazon ECR**.

3. **Amazon EKS** manages the **Kubernetes cluster**.

4. **EC2 worker nodes** run application Pods pulled from **ECR**.

Final Result:



Setting Up the Local Environment

Tools Installed And Verified:

- AWS CLI — sends commands to your AWS account from the terminal
- Docker Desktop — builds images and lets you test containers locally before pushing
- kubectl — the standard Kubernetes CLI for deploying, inspecting, and managing workloads
- eksctl — a utility built for EKS that automates cluster creation, node group setup, IAM roles, and VPC config

```
PS C:\Users\rohan> aws --version
aws-cli/2.34.54 Python/3.14.5 Windows/11 exe/AMD64
PS C:\Users\rohan> docker --version
Docker version 29.1.3, build f52814d
PS C:\Users\rohan> kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Unable to connect to the server: dial tcp 127.0.0.1:50565: connectex: No connection could be made because the target machine actively refused it.
PS C:\Users\rohan> eksctl version
0.226.0
PS C:\Users\rohan>
```

Configuring AWS CLI:

The AWS CLI doesn't know which account to use until you configure it. By setting up access keys and a default region, every command I run after this automatically goes to my account in us-east-1, which has the best free tier coverage.

“aws configure”

```
PS C:\Users\rohan> aws configure
AWS Access Key ID [*****GRWH]: AKIARQHAPIGQX05QP5A5
AWS Secret Access Key [*****6uc1]: J1S8a2jB8eYmrDoxLf5fxT39Si4FZHdHrhzBRd6Y
Default region name [ap-south-2]: us-east-1
Default output format [json]: json
PS C:\Users\rohan>
```

Containerizing the Application:

Kubernetes runs containers, not raw code. Wrapping the app in a Docker image means it will behave identically on my laptop, on an EC2 node, or anywhere else. It also means Kubernetes can pull, run, and restart it automatically without knowing anything about the underlying app.

The Dockerfile:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
~
~
```

Building and Testing Locally

I built the image and ran it locally to make sure it worked before pushing anything to AWS:

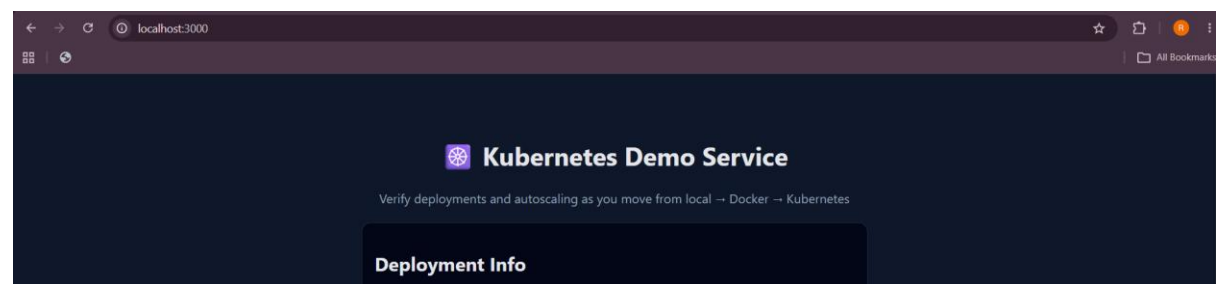
“Docker build -t shopease-app .”

```
Windows PowerShell
PS C:\Users\rohan\OneDrive\Desktop\app> docker run -p 3000:3000 shopease-app

> k8s-demo-service@1.0.0 start
> node src/server.js

Kubernetes Demo Service v1 running on port 3000 (instance: a299c8ba441a)
```

“Docker run -p 3000:3000 shopease-app”



Pushing the Image to Amazon ECR:

EKS worker nodes need to pull the Docker image from somewhere. ECR is AWS's private container registry. Since it lives inside AWS, the cluster pulls images over the internal network without any egress charges, and access is controlled by IAM.

```
PS C:\Users\rohan\OneDrive\Desktop\app> aws ecr create-repository `
>> --repository-name shopease-backend `
>> --region $REGION
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:103549976993:repository/shopease-backend",
    "registryId": "103549976993",
    "repositoryName": "shopease-backend",
    "repositoryUri": "103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend",
    "createdAt": "2026-06-11T11:56:41.743000+05:30",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

Creating the Repository:

```
PS C:\Users\rohan\OneDrive\Desktop\app> aws ecr create-repository `
>> --repository-name shopease-backend `
>> --region $REGION
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:103549976993:repository/shopease-backend",
    "registryId": "103549976993",
    "repositoryName": "shopease-backend",
    "repositoryUri": "103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend",
    "createdAt": "2026-06-11T11:56:41.743000+05:30",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

Logging Docker into ECR:

ECR uses temporary tokens instead of passwords. This command generates a token and pipes it directly to Docker login:

```
PS C:\Users\rohan\OneDrive\Desktop\app> aws ecr get-login-password --region $REGION | docker login --username AWS --pass
word-stdin "$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com"
Login Succeeded
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

Tagging and Pushing:

```
PS C:\Users\rohan\OneDrive\Desktop\app> docker tag shopease-app:latest 103549976993.dkr.ecr.us-east-1.amazonaws.com/shop
ease-backend:v1
PS C:\Users\rohan\OneDrive\Desktop\app> docker push 103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend:v1
The push refers to repository [103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend]
528ecea715d3: Pushed
4feea04c1543: Pushed
dad561a3b5e2: Pushed
bef61ad8152b: Pushed
b523020ecf5c: Pushed
b8461081787b: Pushed
b2cbbfe903b0: Pushed
fff4e2c1b189: Pushed
6a0ac1617861: Pushed
v1: digest: sha256:b8b53161f20d4016a2649f4710f024f2781cf76a384112053f2b3aec2d7f4986 size: 856
PS C:\Users\rohan\OneDrive\Desktop\app> |
```


Creating the Amazon EKS Cluster

EKS gives me a managed Kubernetes control plane. AWS runs and maintains the control plane, I only manage the worker nodes. The control plane handles scheduling, health monitoring, and keeping the desired state running at all times.

Creating the Cluster:

```
2026-06-11 12:18:54 [i] building managed nodegroup stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 12:18:56 [i] deploying stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 12:18:56 [i] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 12:19:27 [i] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 12:20:22 [i] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 12:22:20 [i] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 12:22:20 [i] waiting for the control plane to become ready
2026-06-11 12:22:21 [✓] saved kubeconfig as "C:\\Users\\rohan\\.kube\\config"
2026-06-11 12:22:21 [i] no tasks
2026-06-11 12:22:21 [✓] all EKS cluster resources for "shopease-cluster" have been created
2026-06-11 12:22:22 [i] nodegroup "shopease-nodes" has 2 node(s)
2026-06-11 12:22:22 [i] node "ip-192-168-25-205.ec2.internal" is ready
2026-06-11 12:22:22 [i] node "ip-192-168-63-177.ec2.internal" is ready
2026-06-11 12:22:22 [i] waiting for at least 1 node(s) to become ready in "shopease-nodes"
2026-06-11 12:22:22 [i] nodegroup "shopease-nodes" has 2 node(s)
2026-06-11 12:22:22 [i] node "ip-192-168-25-205.ec2.internal" is ready
2026-06-11 12:22:22 [i] node "ip-192-168-63-177.ec2.internal" is ready
2026-06-11 12:22:22 [✓] created 1 managed nodegroup(s) in cluster "shopease-cluster"
2026-06-11 12:22:24 [i] creating addon: metrics-server
2026-06-11 12:22:24 [i] successfully created addon: metrics-server
2026-06-11 12:22:27 [i] kubectrl command should work with "C:\\Users\\rohan\\.kube\\config", try 'kubectrl get nodes'
2026-06-11 12:22:27 [✓] EKS cluster "shopease-cluster" in "us-east-1" region is ready
PS C:\\Users\\rohan\\OneDrive\\Desktop\\app>
```

Verifying Connectivity:

eksctl automatically updates the local kubeconfig file. I ran this to confirm kubectrl was connected to the cluster:

“kubectrl get nodes”

```
PS C:\\Users\\rohan\\OneDrive\\Desktop\\app> kubectrl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-192-168-25-205.ec2.internal      Ready    <none>    9m7s   v1.34.8-eks-3385e9b
ip-192-168-63-177.ec2.internal      Ready    <none>    9m6s   v1.34.8-eks-3385e9b
PS C:\\Users\\rohan\\OneDrive\\Desktop\\app>
```

Deploying the Application:

A Kubernetes Deployment is how you tell the cluster to run your application. You define how many copies (replicas) you want, which image

to use, and what resources to give each container. Kubernetes then handles the actual scheduling, restarts, and replacement of failed Pods.

The Deployment YAML:

I created a file called deployment.yaml:

```
kind: Deployment
metadata:
  name: shopease-deployment
  labels:
    app: shopease
spec:
  replicas: 2
  selector:
    matchLabels:
      app: shopease
  template:
    metadata:
      labels:
        app: shopease
    spec:
      containers:
        - name: shopease-backend
          image: 103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend:v1
          ports:
            - containerPort: 3000
          resources:
            requests:
              memory: "64Mi"
              cpu: "100m"
            limits:
              memory: "128Mi"
              cpu: "200m"
```

I set replicas: 2 so there is always a backup. If one Pod crashes, the other keeps serving traffic while Kubernetes restarts the failed one. The label app: shopease is what ties everything together the Deployment, the Pods, and the Service all use this label to find each other.

Applying the Deployment:

"kubectl apply -f deployment.yaml"

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl apply -f deployment.yaml
deployment.apps/shopease-deployment created
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

Checking Pod Status:

"kubectl get pods"

“kubectl get deployment shopease-deployment”

“kubectl get pods -o wide”

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
shopease-deployment-5476459f4c-64rtb 1/1     Running   0           9m44s
shopease-deployment-5476459f4c-kkzpg 1/1     Running   0           9m44s
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl get deployment shopease-deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
shopease-deployment 2/2     2             2           9m54s
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE
shopease-deployment-5476459f4c-64rtb 1/1     Running   0           10m   192.168.31.186 ip-192-168-29-15.ec2.internal
shopease-deployment-5476459f4c-kkzpg 1/1     Running   0           10m   192.168.22.39  ip-192-168-29-15.ec2.internal
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

Inspecting Events:

I used kubectl describe on one Pod to see the full lifecycle from image pull to container start:

“kubectl describe pod <pod-name>”

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl describe pod shopease-deployment-5476459f4c-64rtb
Name:                shopease-deployment-5476459f4c-64rtb
Namespace:           default
Priority:             0
Service Account:     default
Node:                ip-192-168-29-15.ec2.internal/192.168.29.15
Start Time:          Thu, 11 Jun 2026 12:41:43 +0530
Labels:              app=shopease
                    pod-template-hash=5476459f4c
Annotations:         <none>
Status:              Running
IP:                  192.168.31.186
IPs:                 IP: 192.168.31.186
Controlled By:       ReplicaSet/shopease-deployment-5476459f4c
Containers:
  shopease-backend:
    Container ID:   containerd://122bcc37ab632066a3d990ff059d20351fcb5f19718f7a138a0bd605eb2820cc
    Image:          103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend:v1
    Image ID:       103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend@sha256:b8b53161f20d4016a2649f4710f024f2781cf76a384112053f2b3aec2d7f4986
    Port:           3000/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Thu, 11 Jun 2026 12:41:48 +0530
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:          200m
      memory:       128Mi
```

Checking Logs:

“kubectl logs -l app=shopease”

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl logs -l app=shopease
> k8s-demo-service@1.0.0 start
> node src/server.js

Kubernetes Demo Service v1 running on port 3000 (instance: shopease-deployment-5476459f4c-64rtb)
> k8s-demo-service@1.0.0 start
> node src/server.js

Kubernetes Demo Service v1 running on port 3000 (instance: shopease-deployment-5476459f4c-kkzpg)
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

Exposing the Application:

Pods get a new IP address every time they restart, so you cannot hardcode their IPs. A Kubernetes Service solves this with a stable endpoint. I used NodePort which exposes the service on a fixed port on each EC2 node. This is the free option LoadBalancer would create an AWS Load Balancer with additional hourly cost.

The Service YAML:

I created a file called service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: shopease-service
spec:
  type: NodePort
  selector:
    app: shopease
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3000
      nodePort: 30080
  ~
  ~
  ~
```

Applying the Service:

“kubectl apply -f service.yml”

“kubectl get service shopease-service”

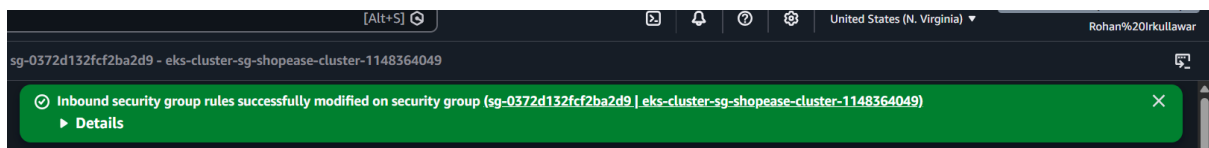
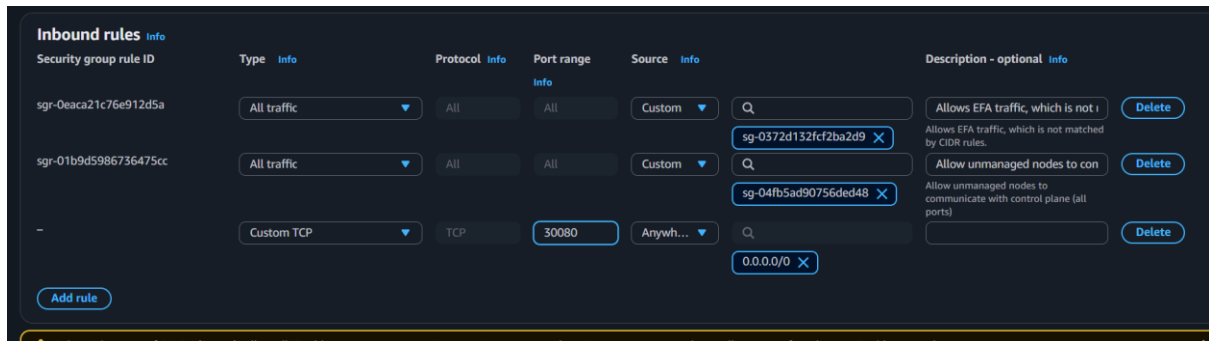
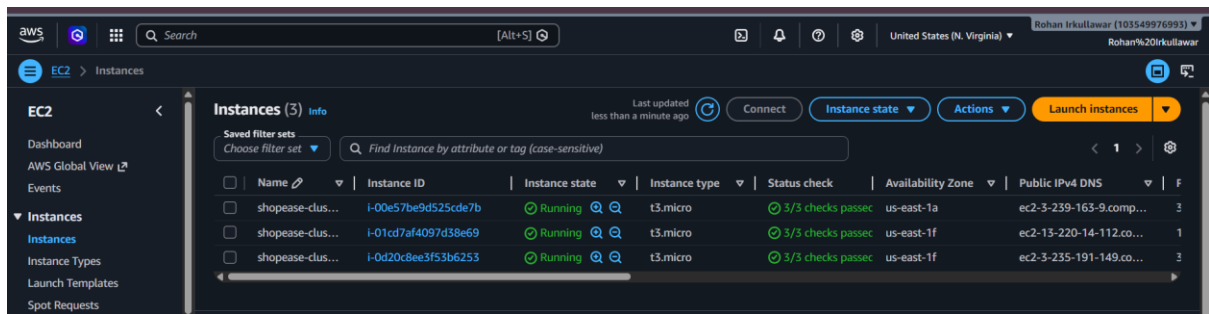
```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl apply -f service.yml
service/shopease-service created
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl get service shopease-service
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
shopease-service    NodePort    10.100.166.88 <none>       80:30080/TCP     29s
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

Opening the Security Group:

EC2 nodes block all inbound traffic by default. Port 30080 needed to be opened manually in the Security Group:

1. AWS Console -> EC2 -> Instances -> clicked one of the worker nodes
2. Scrolled down to the Security tab -> clicked the Security Group link
3. Clicked Edit inbound rules -> Add rule
4. Set: Type = Custom TCP, Port = 30080, Source = 0.0.0.0/0
5. Saved the rules

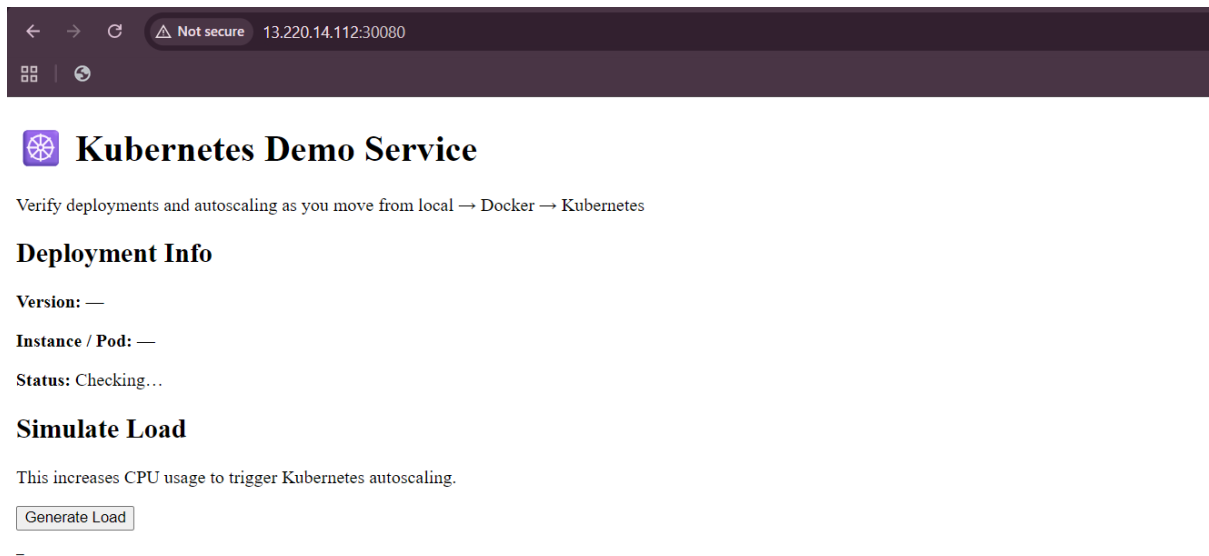


Accessing the Live Application:

I got the public IP of a worker node and opened the app in the browser

“kubectl get nodes -o wide”

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP
ip-192-168-25-205.ec2.internal      Ready    <none>    39m   v1.34.8-eks-3385e9b   192.168.25.205   3.235.191.149
Amazon Linux 2023.11.20260526      6.12.88-119.157.amzn2023.x86_64   containerd://2.2.3+unknown
ip-192-168-29-15.ec2.internal      Ready    <none>    18m   v1.34.8-eks-3385e9b   192.168.29.15    13.220.14.112
Amazon Linux 2023.11.20260526      6.12.88-119.157.amzn2023.x86_64   containerd://2.2.3+unknown
ip-192-168-63-177.ec2.internal      Ready    <none>    39m   v1.34.8-eks-3385e9b   192.168.63.177   3.239.163.9
Amazon Linux 2023.11.20260526      6.12.88-119.157.amzn2023.x86_64   containerd://2.2.3+unknown
PS C:\Users\rohan\OneDrive\Desktop\app>
```



Validating Kubernetes Behavior:

I ran a single command to see every Kubernetes object created in this project:

“**kubectl get all**”

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/shopease-deployment-5476459f4c-64rtb   1/1     Running   0          36m
pod/shopease-deployment-5476459f4c-kkzpg   1/1     Running   0          36m

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/kubernetes                  ClusterIP     10.100.0.1   <none>        443/TCP          55m
service/shopease-service            NodePort      10.100.166.88 <none>        80:30080/TCP     18m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/shopease-deployment  2/2     2            2          36m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/shopease-deployment-5476459f4c  2         2         2       36m
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

Testing Self-Healing:

One of the main things Kubernetes is supposed to do is restart failed Pods automatically. I tested this by deleting a Pod on purpose and watching Kubernetes bring a replacement up:

Deleted pod manually

“**kubectl delete pod <pod-name>**”

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl delete pods shopease-deployment-5476459f4c-64rtb
pod "shopease-deployment-5476459f4c-64rtb" deleted from default namespace
PS C:\Users\rohan\OneDrive\Desktop\app> |
```

Watch the replacement come up

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl get pods -w
NAME                                READY   STATUS             RESTARTS   AGE
shopease-deployment-5476459f4c-cfjmc 0/1     ContainerCreating   0          32s
shopease-deployment-5476459f4c-kkzpg 1/1     Running             0          40m
shopease-deployment-5476459f4c-cfjmc 1/1     Running             0          40s
```

The replacement Pod was running in under 30 seconds. This is the desired state reconciliation loop in action Kubernetes noticed the replica count dropped to 1 and immediately scheduled a new Pod to bring it back to 2.

Cleanup:

EKS charges roughly \$0.10 per hour for the control plane, even when no workloads are running. Deleting the cluster the same day keeps the total cost well under \$1. I deleted Kubernetes resources first, then the cluster, then ECR.

Removing Kubernetes resources:

“`kubectl delete -f service.yml`”

“`kubectl delete -f deployment.yml`”

```
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl delete -f service.yml
service "shopease-service" deleted from default namespace
PS C:\Users\rohan\OneDrive\Desktop\app> kubectl delete -f deployment.yml
deployment.apps "shopease-deployment" deleted from default namespace
PS C:\Users\rohan\OneDrive\Desktop\app> eksctl delete cluster --name shopease-cluster --region us-east-1
```


Deleting the EKS cluster (removes EC2 nodes and VPC too):

"eksctl delete cluster --name shopease-cluster --region us-east-1"

```
PS C:\Users\rohan\OneDrive\Desktop\app> eksctl delete cluster --name shopease-cluster --region us-east-1
2026-06-11 13:47:03 [I] deleting EKS cluster "shopease-cluster"
2026-06-11 13:47:05 [I] will drain 0 unmanaged nodegroup(s) in cluster "shopease-cluster"
2026-06-11 13:47:05 [I] starting parallel draining, max in-flight of 1
2026-06-11 13:47:07 [I] deleted 0 Fargate profile(s)
2026-06-11 13:47:09 [✓] kubeconfig has been updated
2026-06-11 13:47:09 [I] cleaning up AWS load balancers created by Kubernetes objects of Kind Service, Ingress, or Gateway
2026-06-11 13:47:15 [I]
2 sequential tasks: { delete nodegroup "shopease-nodes", delete cluster control plane "shopease-cluster" [async]
}
2026-06-11 13:47:16 [I] will delete stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:47:16 [I] waiting for stack "eksctl-shopease-cluster-nodegroup-shopease-nodes" to get deleted
2026-06-11 13:47:16 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:47:47 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:48:26 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:49:18 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:50:09 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:51:40 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:52:15 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:52:55 [I] waiting for CloudFormation stack "eksctl-shopease-cluster-nodegroup-shopease-nodes"
2026-06-11 13:54:27 [I] will delete stack "eksctl-shopease-cluster-cluster"
2026-06-11 13:54:28 [✓] all cluster resources were deleted
```

Deleting the Repository:

"aws ecr delete-repository `

--repository-name shopease-backend `

--region us-east-1 `

--force"

```
PS C:\Users\rohan\OneDrive\Desktop\app> aws ecr delete-repository `
>> --repository-name shopease-backend `
>> --region us-east-1 `
>> --force
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:103549976993:repository/shopease-backend",
    "registryId": "103549976993",
    "repositoryName": "shopease-backend",
    "repositoryUri": "103549976993.dkr.ecr.us-east-1.amazonaws.com/shopease-backend",
    "createdAt": "2026-06-11T11:56:41.743000+05:30",
    "imageTagMutability": "MUTABLE"
  }
}
```

